



```
In [3]: # Importing libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.impute import SimpleImputer

from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier

from sklearn.metrics import accuracy_score
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report

import warnings
warnings.filterwarnings('ignore')
```

```
In [4]: # Importing data

df = pd.read_csv("C:/Users/Prabhakar/Desktop/milan excel practice sheets/pytho
```

```
In [5]: df.shape
```

```
Out[5]: (30000, 7)
```

```
In [6]: df.head(10)
```

```
Out[6]:
```

	Patient ID	Age	Gender	Disease	Blood Pressure	Sugar Level	BMI
0	P16212	92.0	Male	Heart Disease	120/80	145.0	17.9
1	P24210	46.0	Other	Flu	NaN	NaN	NaN
2	P21289	84.0	Male	Flu	140/90	200.0	25.9
3	P19015	16.0	Male	Healthy	NaN	194.0	21.4
4	P25687	50.0	Male	NaN	140/90	293.0	22.6
5	P17234	93.0	Other	Flu	160/100	106.0	22.4
6	P24587	85.0	Other	Hypertension	NaN	102.0	26.8
7	P25045	75.0	NaN	Flu	160/100	103.0	25.3
8	P21747	61.0	Female	Diabetes	110/70	212.0	21.9
9	P12826	23.0	Male	Hypertension	120/80	235.0	21.1

```
In [7]: df.info()
```

```

<class 'pandas.DataFrame'>
RangeIndex: 30000 entries, 0 to 29999
Data columns (total 7 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Patient ID            30000 non-null  str
1   Age                   28547 non-null  float64
2   Gender                22383 non-null  str
3   Disease               26147 non-null  str
4   Blood Pressure        19906 non-null  str
5   Sugar Level           27562 non-null  float64
6   BMI                   27933 non-null  float64
dtypes: float64(3), str(4)
memory usage: 1.6 MB

```

```
In [8]: df.isnull().sum()
```

```

Out[8]: Patient ID      0
        Age            1453
        Gender         7617
        Disease        3853
        Blood Pressure 10094
        Sugar Level    2438
        BMI            2067
        dtype: int64

```

```
In [9]: df.duplicated().sum()
```

```
Out[9]: np.int64(0)
```

```
In [10]: df.describe()
```

```

Out[10]:
```

	Age	Sugar Level	BMI
count	28547.000000	27562.000000	27933.000000
mean	47.661436	185.108410	27.498439
std	27.465294	66.581596	7.215704
min	1.000000	70.000000	15.000000
25%	24.000000	127.000000	21.300000
50%	47.000000	185.000000	27.500000
75%	71.000000	243.000000	33.700000
max	95.000000	300.000000	40.000000

```
In [12]: df['Age'] = df['Age'].astype('Int64')
```

```
In [13]: df.describe()
```

Out[13]:

	Age	Sugar Level	BMI
count	28547.0	27562.000000	27933.000000
mean	47.661436	185.108410	27.498439
std	27.465294	66.581596	7.215704
min	1.0	70.000000	15.000000
25%	24.0	127.000000	21.300000
50%	47.0	185.000000	27.500000
75%	71.0	243.000000	33.700000
max	95.0	300.000000	40.000000

In [14]: `df.columns`

Out[14]: Index(['Patient ID', 'Age', 'Gender', 'Disease', 'Blood Pressure',
 'Sugar Level', 'BMI'],
 dtype='str')

In [15]: `df_clean = df.copy()`

```
In [16]: #Data Cleaning:
numerical_cols = ['Age', 'Sugar Level', 'BMI']
for col in numerical_cols:
    df_clean[col] = pd.to_numeric(df_clean[col],errors='coerce')

# Convert Blood Pressure into Systolic BP and Diastolic BP :
bp_split = df_clean['Blood Pressure'].astype(str).str.split('/', expand=True)
df_clean['Systolic BP'] = pd.to_numeric(bp_split[0], errors='coerce')
df_clean['Diastolic BP'] = pd.to_numeric(bp_split[1],errors='coerce')

#Fill missing values:
df_clean.drop(columns=['Blood Pressure'],inplace=True)
numerical_cols = ['Age', 'Sugar Level', 'BMI', 'Systolic BP', 'Diastolic BP']
for col in numerical_cols:
    df_clean[col] = df_clean[col].fillna(
        df_clean[col].median()
    )
```

```
In [17]: # Fill category missing values:
categorical_cols = ['Gender', 'Disease']
for col in categorical_cols:
    df_clean[col] = df_clean[col].fillna(
        df_clean[col].mode()[0]
    )
```

```
In [18]: # Check missing values
print(df_clean.isnull().sum())
```

```
# Check data types
print(df_clean.dtypes)
```

```
Patient ID      0
Age             0
Gender          0
Disease         0
Sugar Level     0
BMI            0
Systolic BP     0
Diastolic BP    0
dtype: int64
Patient ID      str
Age            Int64
Gender         str
Disease        str
Sugar Level    float64
BMI           float64
Systolic BP    float64
Diastolic BP   float64
dtype: object
```

```
In [19]: # Create a new Blood Pressure Category column:
def classify_blood_pressure(row):
    systolic = row['Systolic BP']
    diastolic = row['Diastolic BP']

    # Normal Blood Pressure:
    if (115 <= systolic <= 129) and (75 <= diastolic <= 85):
        return 'Normal'
    # High Blood Pressure:
    elif (systolic > 129) or (diastolic > 85):
        return 'High'
    # Low Blood Pressure:
    elif (systolic < 115) or (diastolic < 75):
        return 'Low'
    else:
        return 'Unclassified'

df_clean['Blood Pressure Category'] = df_clean.apply(classify_blood_pressure, axis=1)
df_clean[
    ['Systolic BP', 'Diastolic BP', 'Blood Pressure Category']
].head(10)
```

Out[19]:

	Systolic BP	Diastolic BP	Blood Pressure Category
0	120.0	80.0	Normal
1	120.0	80.0	Normal
2	140.0	90.0	High
3	120.0	80.0	Normal
4	140.0	90.0	High
5	160.0	100.0	High
6	120.0	80.0	Normal
7	160.0	100.0	High
8	110.0	70.0	Low
9	120.0	80.0	Normal

0	120.0	80.0	Normal
1	120.0	80.0	Normal
2	140.0	90.0	High
3	120.0	80.0	Normal
4	140.0	90.0	High
5	160.0	100.0	High
6	120.0	80.0	Normal
7	160.0	100.0	High
8	110.0	70.0	Low
9	120.0	80.0	Normal

```
In [20]: df_clean['Blood Pressure Category'].value_counts(dropna=False)
```

Out[20]: Blood Pressure Category
Normal 15103
High 9933
Low 4964
Name: count, dtype: int64

```
In [21]: #Remove duplicate
duplicates = df_clean.duplicated().sum()
print("Duplicate Rows Before:", duplicates)
```

Duplicate Rows Before: 0

```
In [22]: df_clean.drop_duplicates(inplace=True)
print("Duplicate Rows After:", df_clean.duplicated().sum())
```

Duplicate Rows After: 0

```
In [23]: #IQR Method(outliers removal)
def remove_outliers(data, column):
    Q1 = data[column].quantile(0.25)
    Q3 = data[column].quantile(0.75)

    IQR = Q3 - Q1
    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR
    return data[(data[column] >= lower) & (data[column] <= upper)]

for col in numerical_cols:
    df_clean = remove_outliers(df_clean, col)

print("Dataset Shape After Removing Outliers:")
print(df_clean.shape)
```

Dataset Shape After Removing Outliers:
(30000, 9)

```
In [24]: df_clean.head()
```

```
Out[24]:
```

	Patient ID	Age	Gender	Disease	Sugar Level	BMI	Systolic BP	Diastolic BP	Blood Pressure Category
0	P16212	92	Male	Heart Disease	145.0	17.9	120.0	80.0	Normal
1	P24210	46	Other	Flu	185.0	27.5	120.0	80.0	Normal
2	P21289	84	Male	Flu	200.0	25.9	140.0	90.0	High
3	P19015	16	Male	Healthy	194.0	21.4	120.0	80.0	Normal
4	P25687	50	Male	Diabetes	293.0	22.6	140.0	90.0	High

```
In [25]: df_clean.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 30000 entries, 0 to 29999
Data columns (total 9 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Patient ID                           30000 non-null  str
1   Age                                   30000 non-null  Int64
2   Gender                               30000 non-null  str
3   Disease                              30000 non-null  str
4   Sugar Level                           30000 non-null  float64
5   BMI                                   30000 non-null  float64
6   Systolic BP                           30000 non-null  float64
7   Diastolic BP                           30000 non-null  float64
8   Blood Pressure Category               30000 non-null  str
dtypes: Int64(1), float64(4), str(4)
memory usage: 2.1 MB
```

```
In [26]: df_clean.to_csv("Healthcare_Dataset_Cleaned.csv", index=False)
print("Cleaned dataset saved successfully.")
```

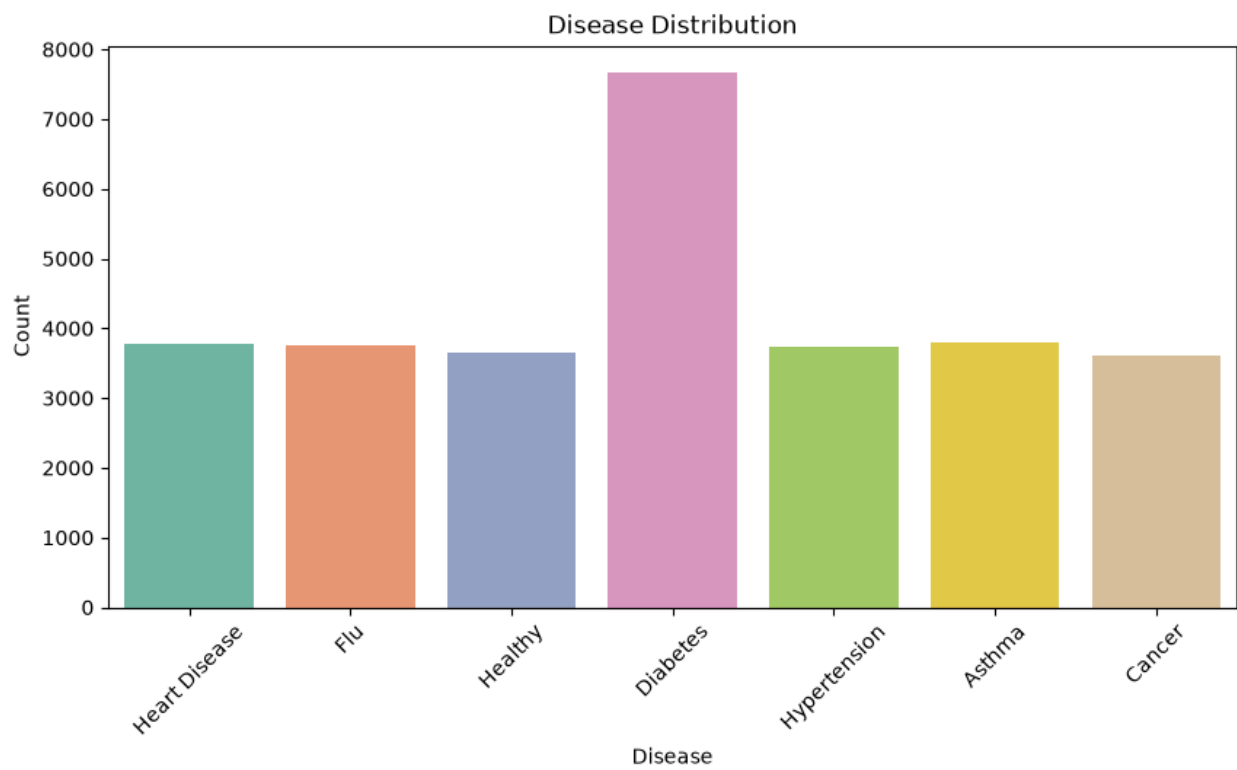
Cleaned dataset saved successfully.

```
In [27]: df_clean.describe(include='all')
```

Out[27]:

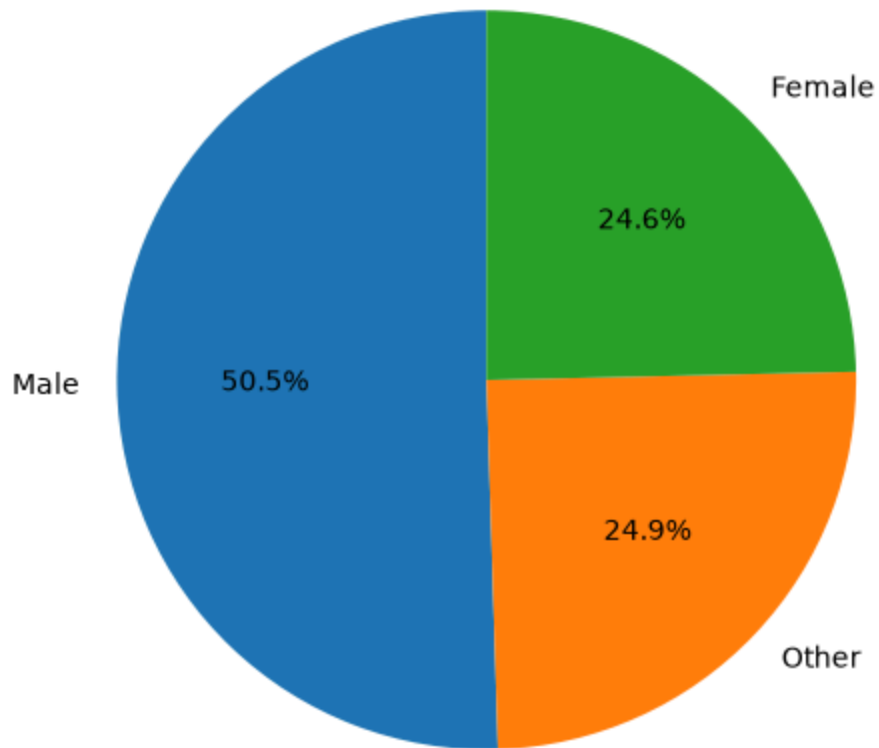
	Patient ID	Age	Gender	Disease	Sugar Level	BMI	Syst
count	30000	30000.0	30000	30000	30000.000000	30000.000000	30000
unique	15235	<NA>	3	7	NaN	NaN	
top	P22897	<NA>	Male	Diabetes	NaN	NaN	
freq	10	<NA>	15148	7671	NaN	NaN	
mean	NaN	47.6294	NaN	NaN	185.099600	27.498547	128
std	NaN	26.792275	NaN	NaN	63.818757	6.962678	16
min	NaN	1.0	NaN	NaN	70.000000	15.000000	110
25%	NaN	25.0	NaN	NaN	132.000000	21.700000	120
50%	NaN	47.0	NaN	NaN	185.000000	27.500000	120
75%	NaN	70.0	NaN	NaN	237.000000	33.300000	140
max	NaN	95.0	NaN	NaN	300.000000	40.000000	160

```
In [28]: #Disease distribution:
plt.figure(figsize=(10,5))
sns.countplot(x='Disease', data=df_clean, palette='Set2')
plt.title("Disease Distribution")
plt.xticks(rotation=45)
plt.xlabel("Disease")
plt.ylabel("Count")
plt.show()
```

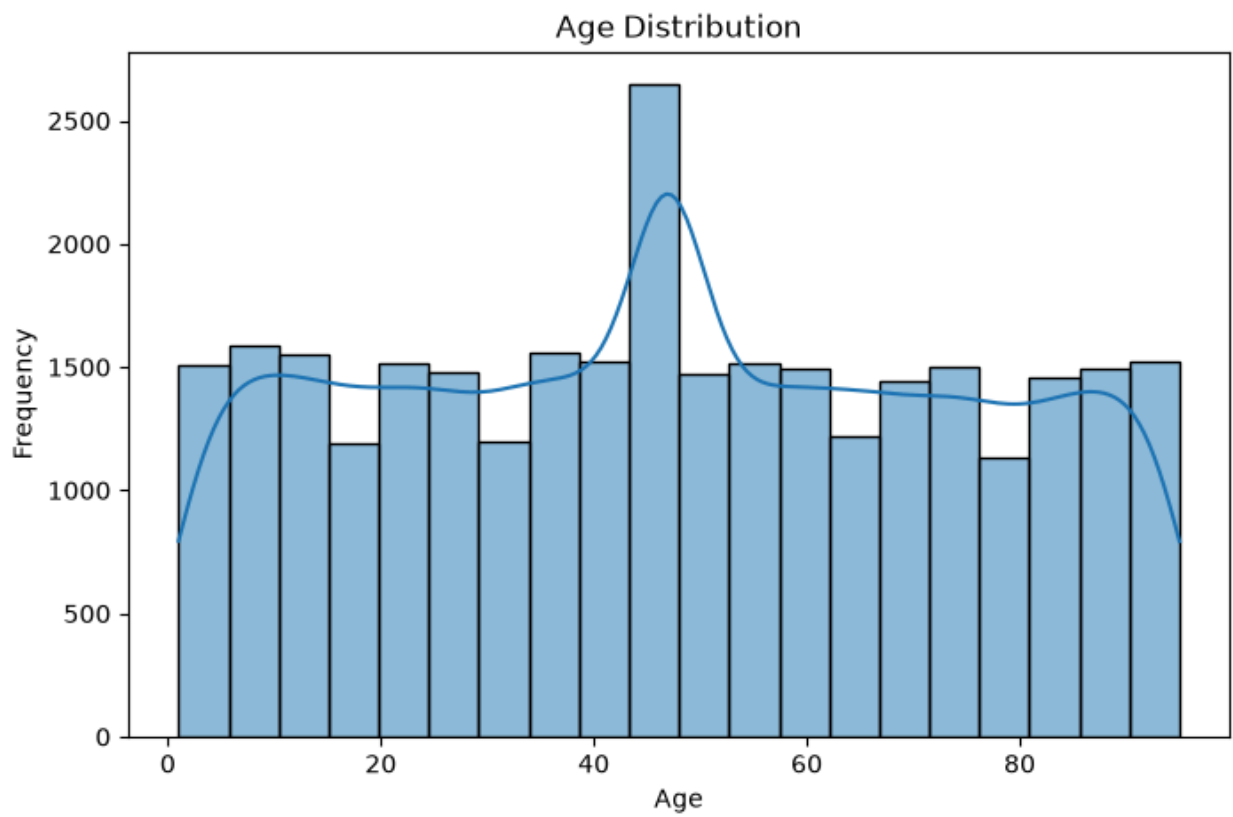


```
In [29]: #Gender Distribution:
plt.figure(figsize=(6,6))
df_clean['Gender'].value_counts().plot(kind='pie',autopct='%1.1f%%',startangle=
plt.title("Gender Distribution")
plt.ylabel("")
plt.show()
```

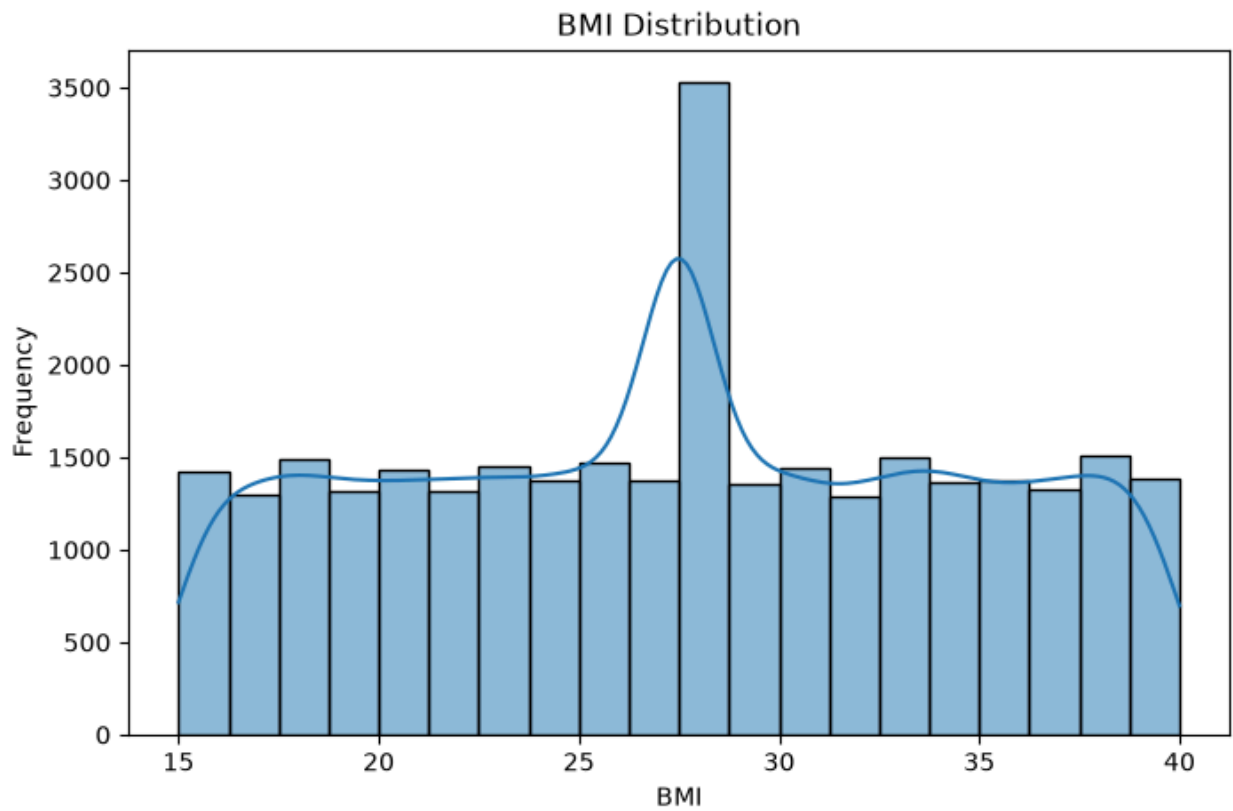

Gender Distribution



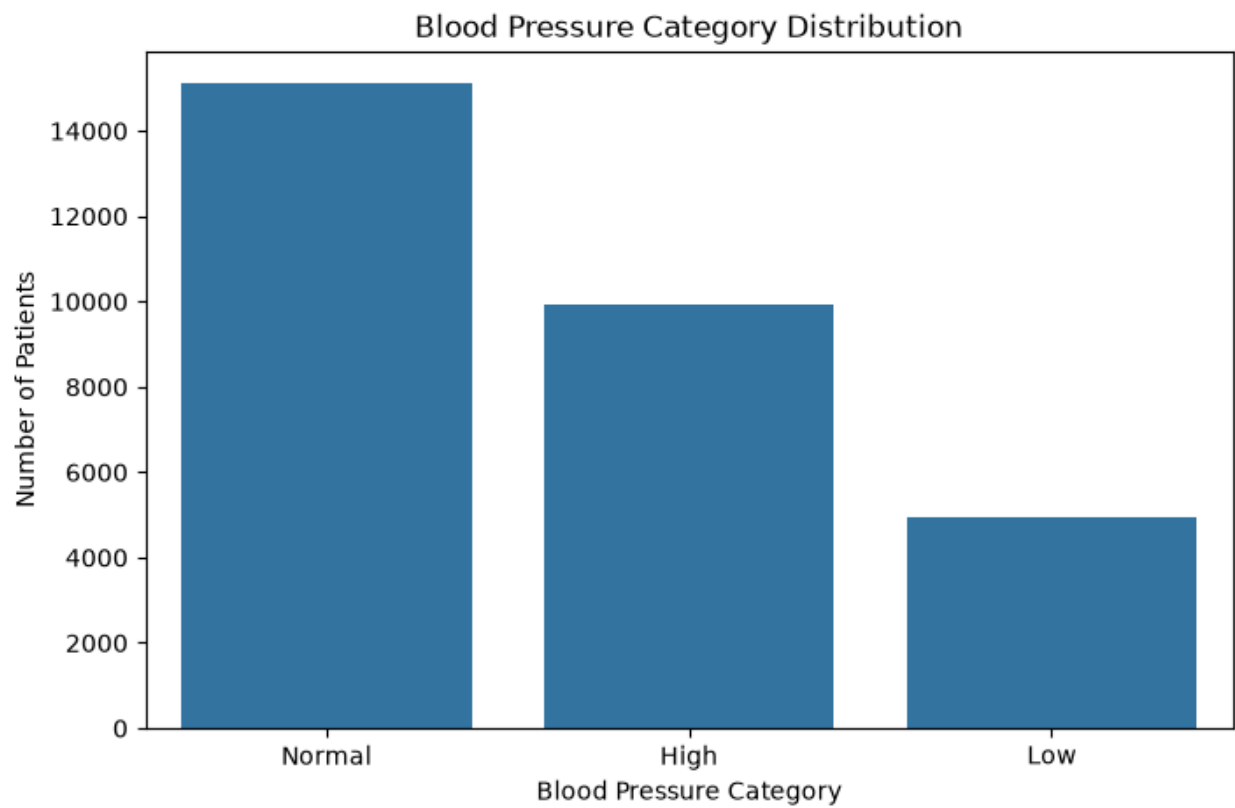
```
In [30]: #Age distribution
plt.figure(figsize=(8,5))
sns.histplot(df_clean['Age'], bins=20, kde=True)
plt.title("Age Distribution")
plt.xlabel("Age")
plt.ylabel("Frequency")
plt.show()
```



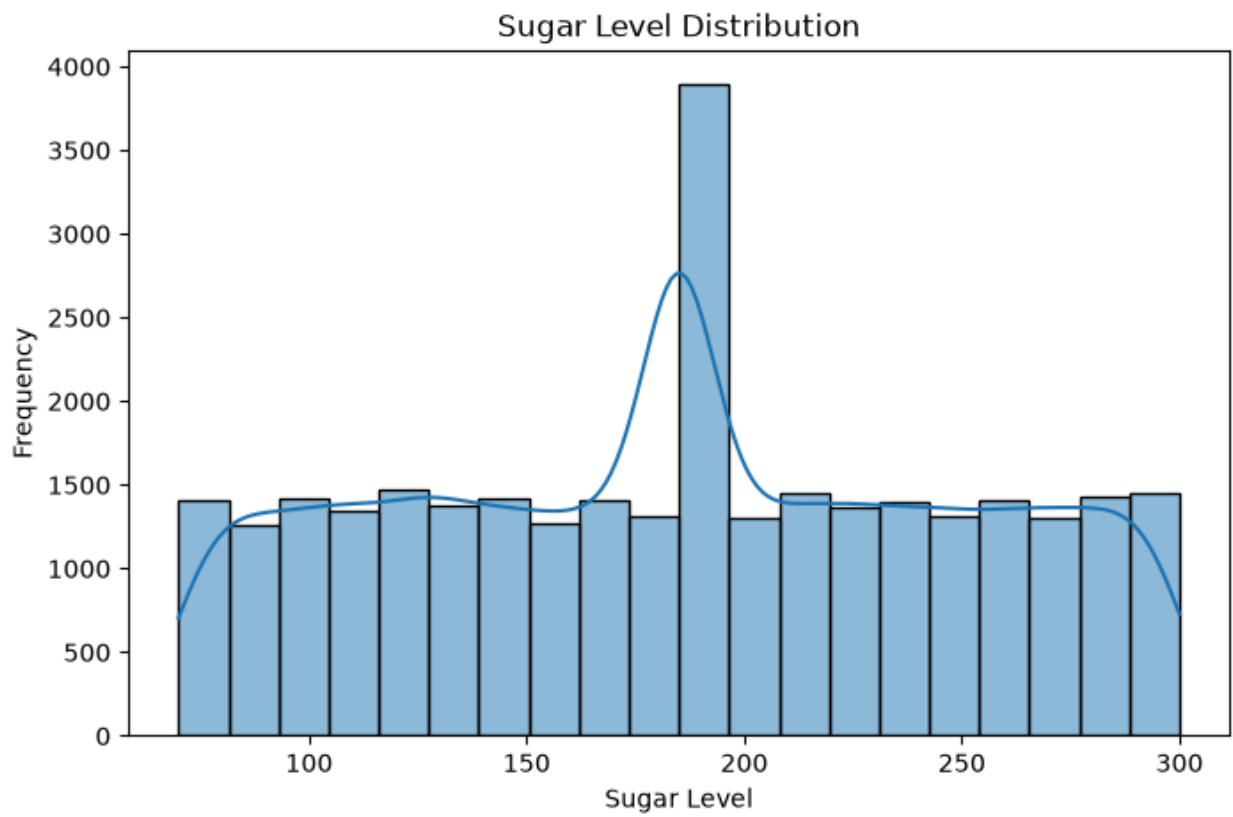
```
In [31]: #BMI Distribution:
plt.figure(figsize=(8,5))
sns.histplot(df_clean['BMI'], bins=20, kde=True)
plt.title("BMI Distribution")
plt.xlabel("BMI")
plt.ylabel("Frequency")
plt.show()
```



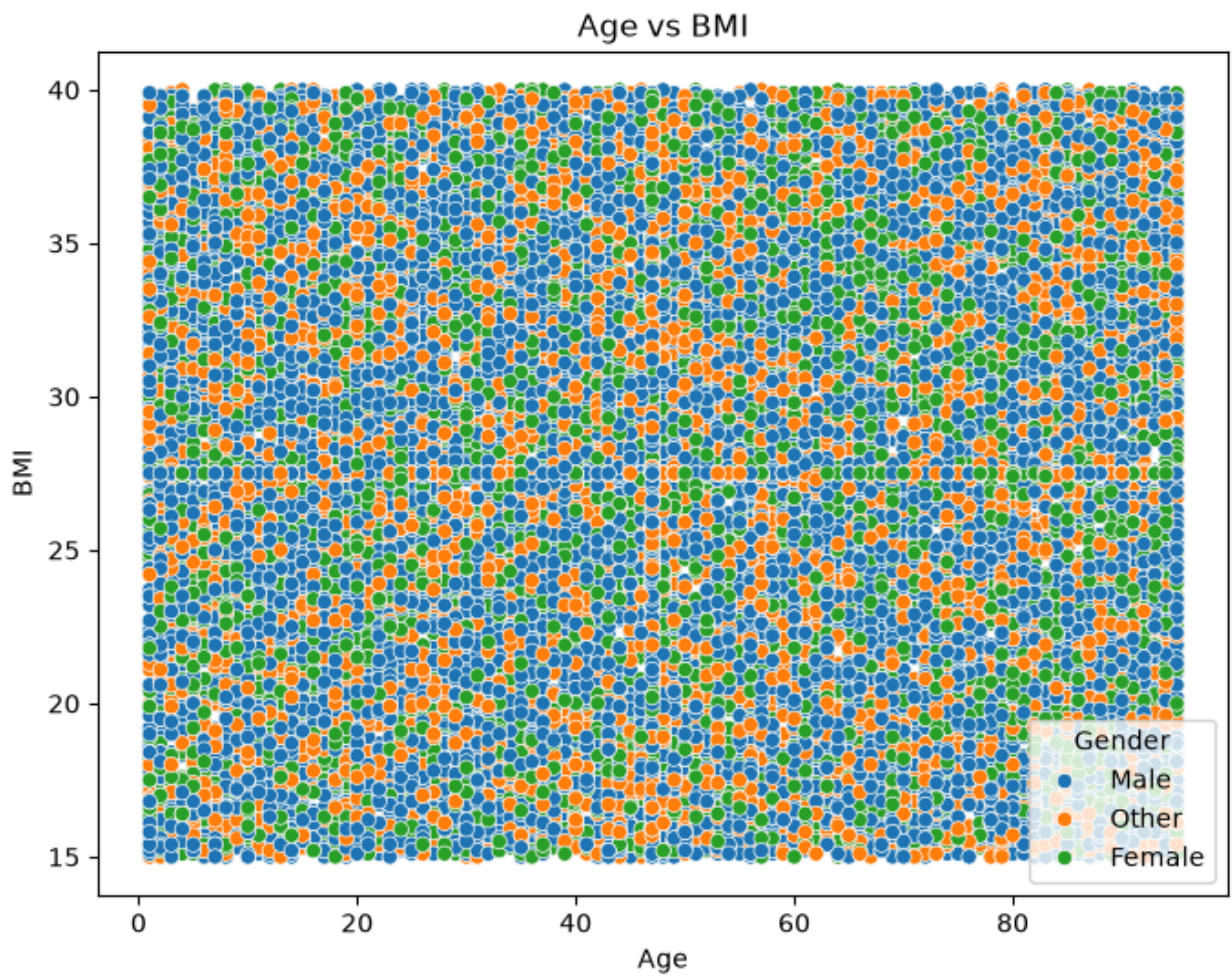
```
In [32]: #Blood Pressure Category Distribution:
plt.figure(figsize=(8,5))
sns.countplot(data=df_clean,x='Blood Pressure Category')
plt.title("Blood Pressure Category Distribution")
plt.xlabel("Blood Pressure Category")
plt.ylabel("Number of Patients")
plt.show()
```



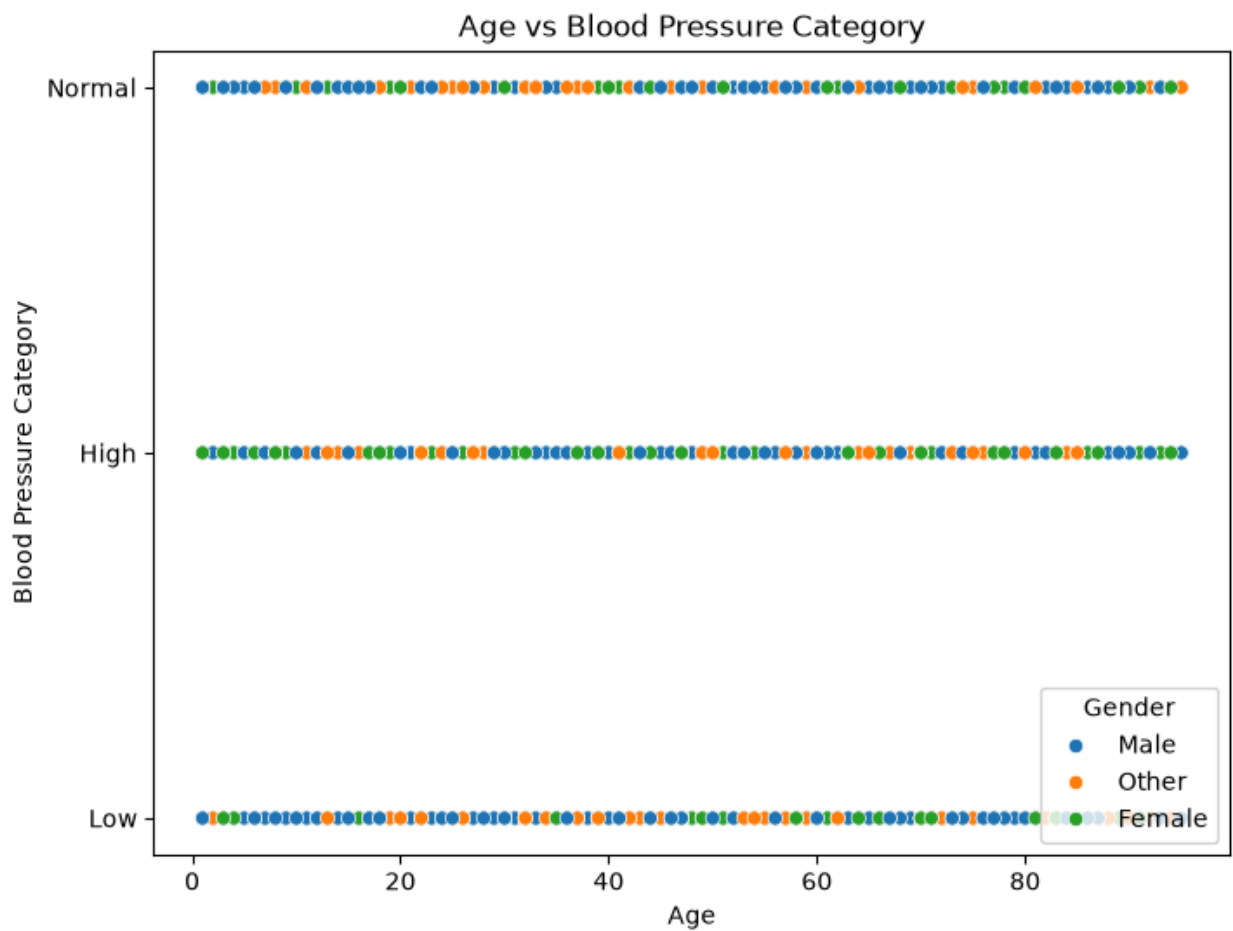
```
In [33]: #Sugar level Distribution:
plt.figure(figsize=(8,5))
sns.histplot(df_clean['Sugar Level'], bins=20, kde=True)
plt.title("Sugar Level Distribution")
plt.xlabel("Sugar Level")
plt.ylabel("Frequency")
plt.show()
```



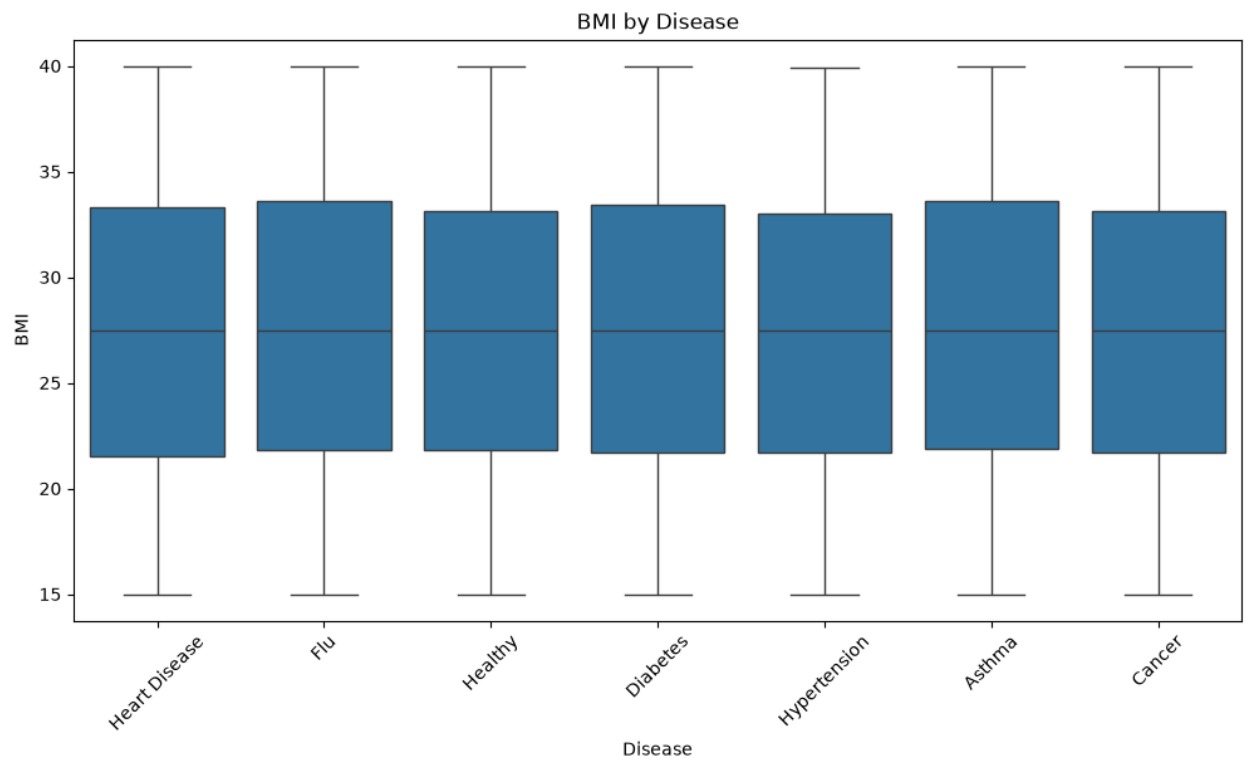
```
In [34]: #Age Vs BMI:
plt.figure(figsize=(8,6))
sns.scatterplot(x='Age',y='BMI',hue='Gender',data=df_clean)
plt.title("Age vs BMI")
plt.show()
```



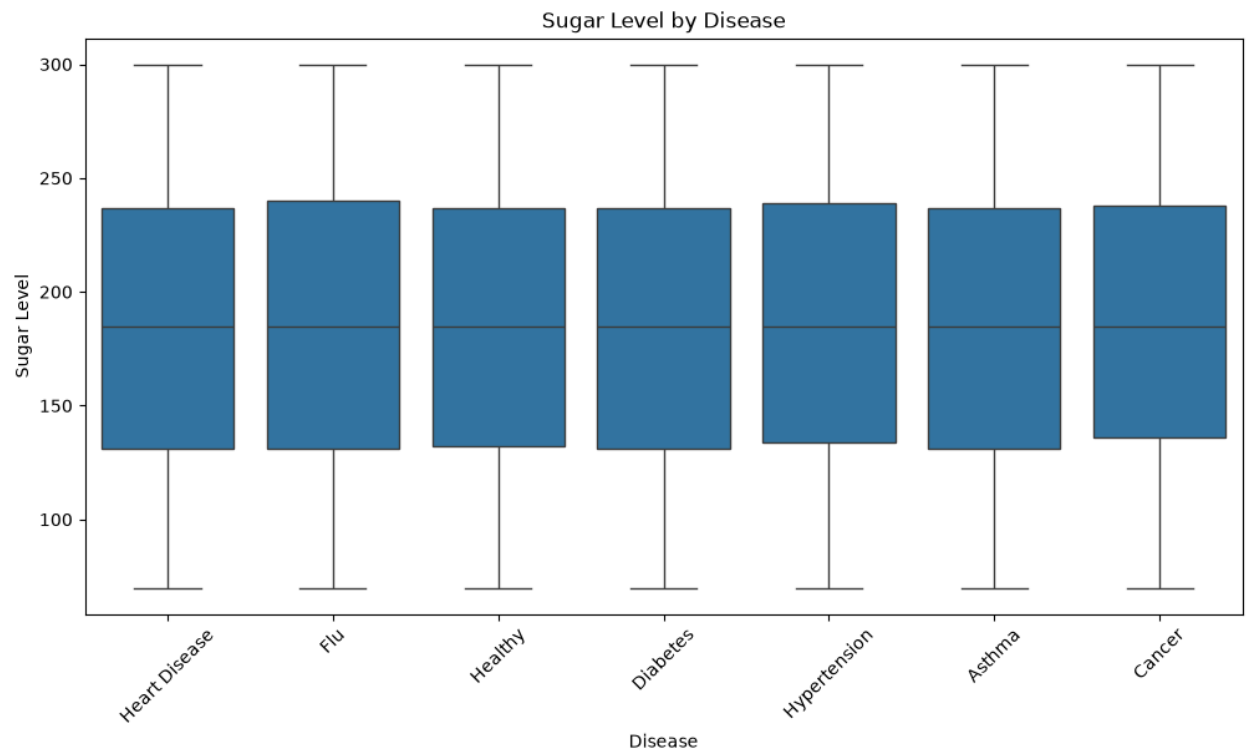
```
In [35]: #Age vs Blood pressure category:
plt.figure(figsize=(8,6))
sns.scatterplot(x='Age',y='Blood Pressure Category',hue='Gender',data=df_clear)
plt.title("Age vs Blood Pressure Category")
plt.show()
```



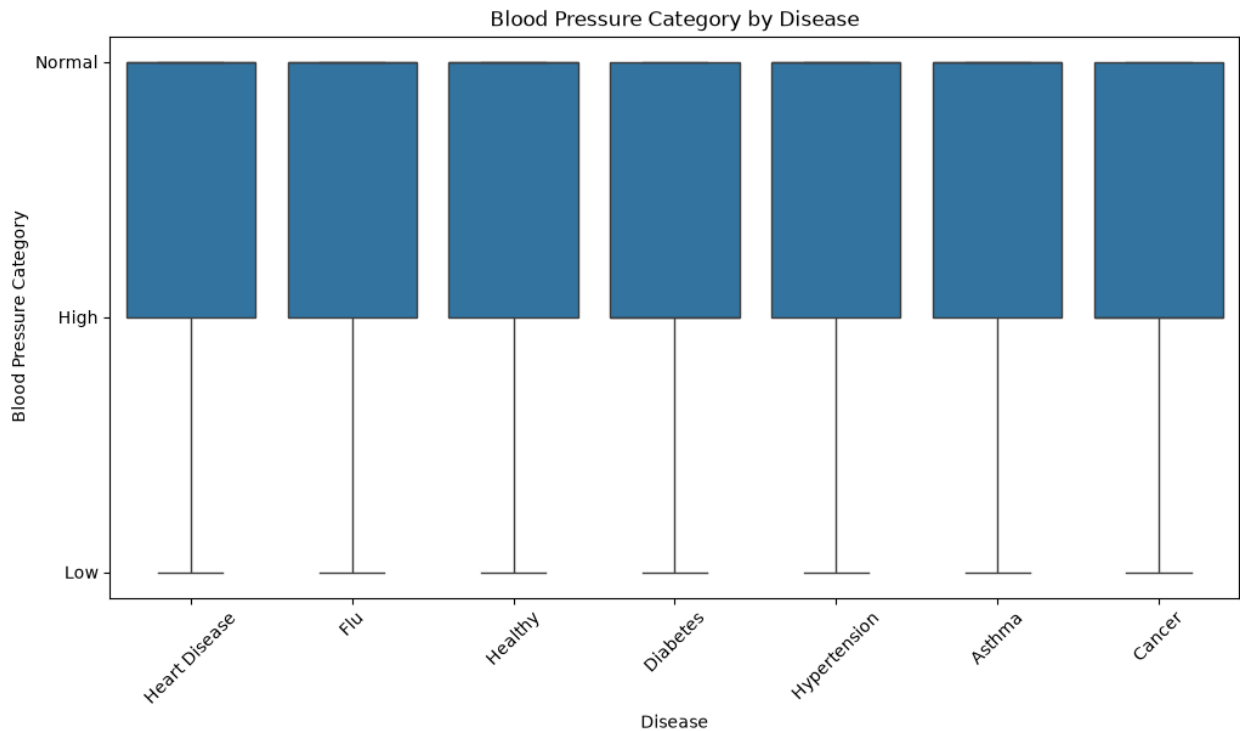
```
In [36]: #BMI Vs Disease:
plt.figure(figsize=(12,6))
sns.boxplot(x='Disease',y='BMI',data=df_clean)
plt.xticks(rotation=45)
plt.title("BMI by Disease")
plt.show()
```



```
In [37]: #Sugar level Vs Disease:
plt.figure(figsize=(12,6))
sns.boxplot(x='Disease',y='Sugar Level',data=df_clean)
plt.xticks(rotation=45)
plt.title("Sugar Level by Disease")
plt.show()
```

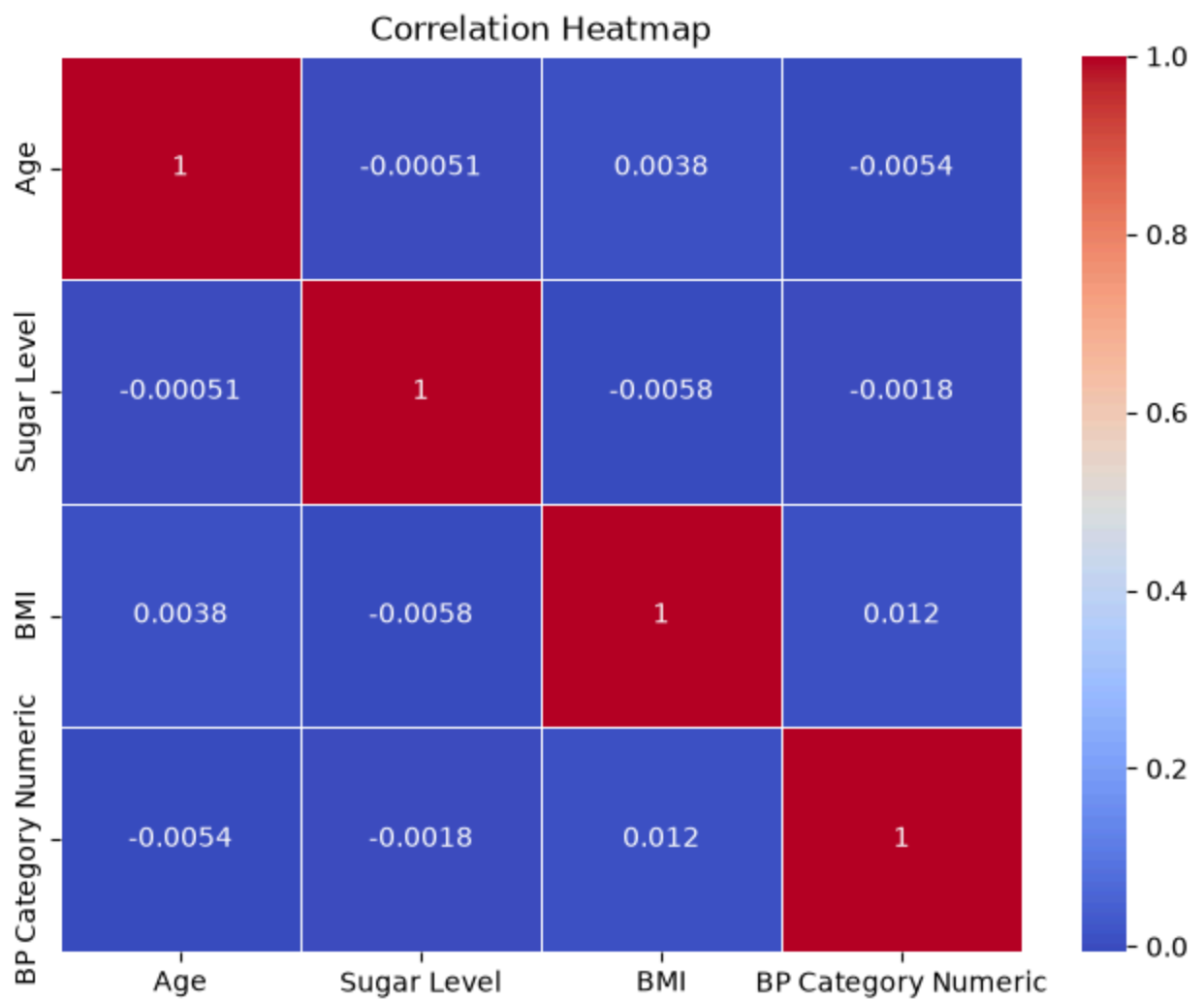



```
In [38]: #Blood Pressure category vs disease:
plt.figure(figsize=(12,6))
sns.boxplot(x='Disease',y='Blood Pressure Category',data=df_clean)
plt.xticks(rotation=45)
plt.title("Blood Pressure Category by Disease")
plt.show()
```

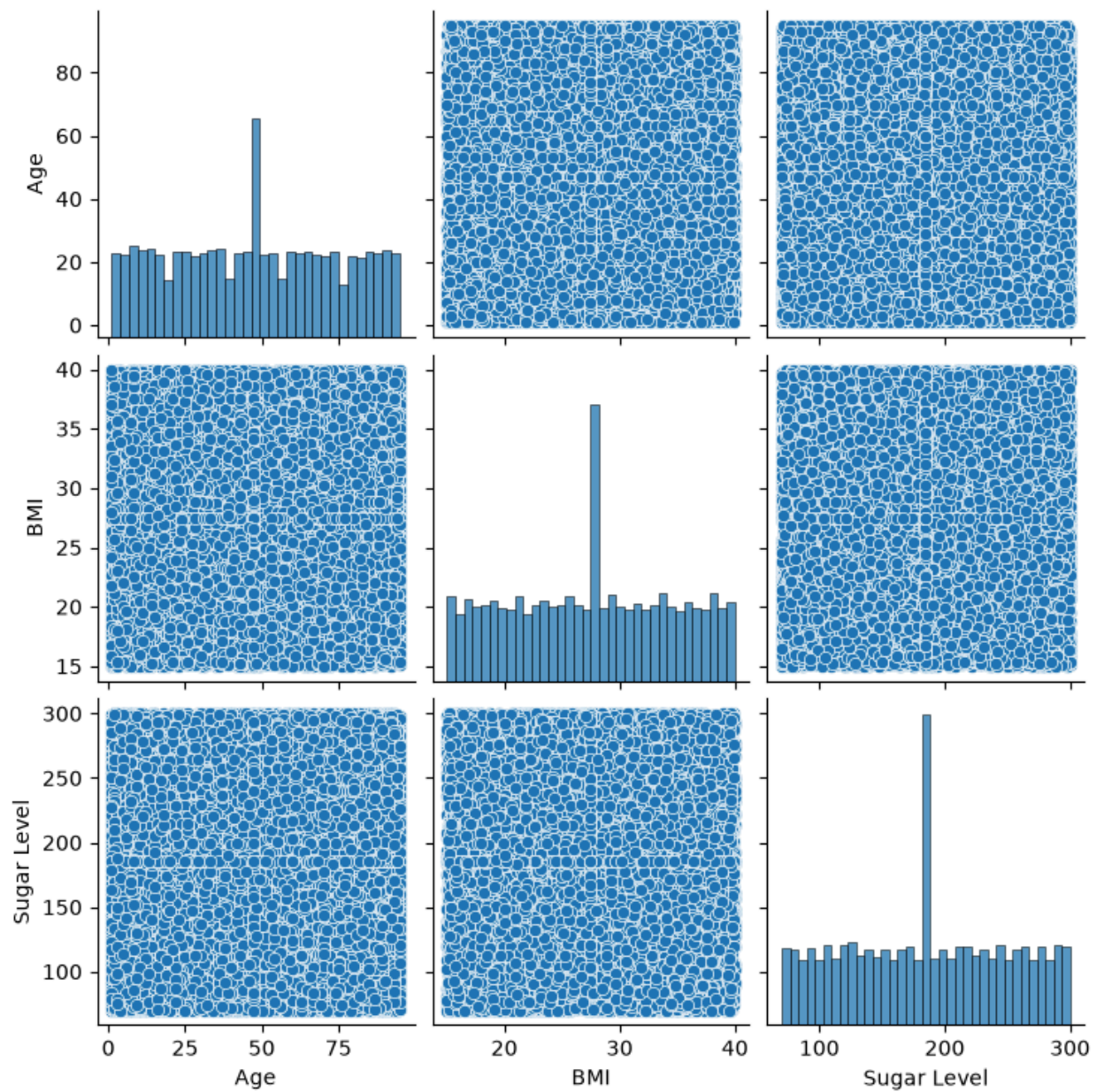


```
In [39]: # Convert Blood Pressure Category into numerical values
df_clean['BP Category Numeric'] = df_clean['Blood Pressure Category'].map({'Low': 1, 'High': 2, 'Normal': 3})
# Select columns for correlation
numeric_df = df_clean[
    ['Age', 'Sugar Level', 'BMI', 'BP Category Numeric']
]
```

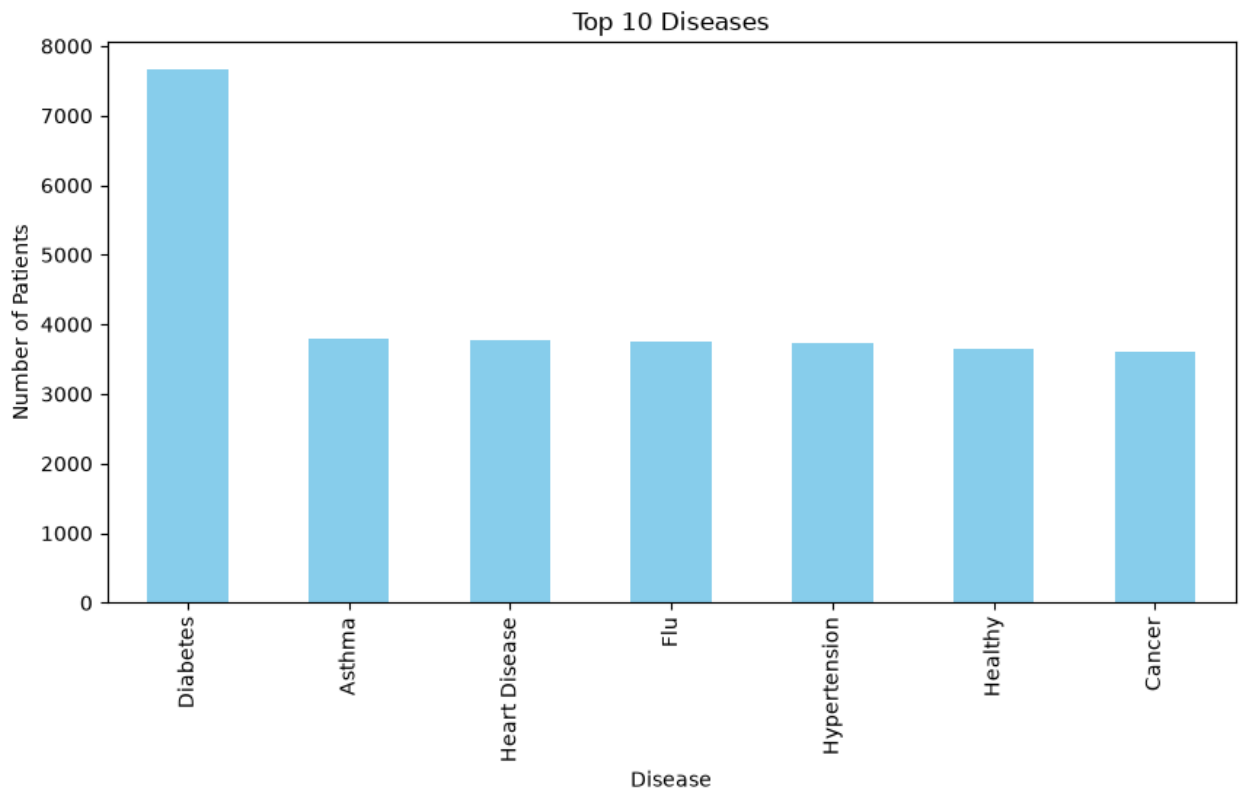
```
In [40]: #Correlation:
plt.figure(figsize=(8, 6))
sns.heatmap(numeric_df.corr(),annot=True,cmap='coolwarm',linewidths=0.5)
plt.title("Correlation Heatmap")
plt.show()
```



```
In [41]: sns.pairplot(df_clean[['Age', 'BMI', 'Blood Pressure Category', 'Sugar Level']])  
plt.show()
```



```
In [43]: #TOP 10 Disease:
plt.figure(figsize=(10,5))
df_clean['Disease'].value_counts().head(10).plot(kind='bar',color='skyblue')
plt.title("Top 10 Diseases")
plt.xlabel("Disease")
plt.ylabel("Number of Patients")
plt.show()
```



In [44]: *#Machine learning: predicting patient disease using age, bmi, sugar level, bp*

```
from sklearn.preprocessing import LabelEncoder
le_gender = LabelEncoder()
le_disease = LabelEncoder()
df_ml = df_clean.copy()
df_ml['Gender'] = le_gender.fit_transform(df_ml['Gender'])
df_ml['Disease'] = le_disease.fit_transform(df_ml['Disease'])
df_ml.head()
```

Out[44]:

	Patient ID	Age	Gender	Disease	Sugar Level	BMI	Systolic BP	Diastolic BP	Blood Pressure Category	Ca N
0	P16212	92	1	5	145.0	17.9	120.0	80.0	Normal	
1	P24210	46	2	3	185.0	27.5	120.0	80.0	Normal	
2	P21289	84	1	3	200.0	25.9	140.0	90.0	High	
3	P19015	16	1	4	194.0	21.4	120.0	80.0	Normal	
4	P25687	50	1	2	293.0	22.6	140.0	90.0	High	

```
In [45]: X = df_ml[['Age', 'Gender', 'BP Category Numeric', 'Sugar Level', 'BMI']]
y = df_ml['Disease']
print(X.head())
print(y.head())
```

	Age	Gender	BP	Category	Numeric	Sugar	Level	BMI
0	92	1			2		145.0	17.9
1	46	2			2		185.0	27.5
2	84	1			3		200.0	25.9
3	16	1			2		194.0	21.4
4	50	1			3		293.0	22.6
0	5							
1	3							
2	3							
3	4							
4	2							

Name: Disease, dtype: int64

```
In [47]: #train_test_splitting:
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X,y,test_size=0.20,random_
print("Training Samples :", X_train.shape)
print("Testing Samples :", X_test.shape)
```

Training Samples : (24000, 5)
Testing Samples : (6000, 5)

```
In [48]: #Logistic regression:
from sklearn.linear_model import LogisticRegression
lr = LogisticRegression(max_iter=1000)
lr.fit(X_train, y_train)
lr_pred = lr.predict(X_test)
```

```
In [49]: #accuracy:
from sklearn.metrics import accuracy_score
lr_acc = accuracy_score(y_test, lr_pred)
print("Logistic Regression Accuracy :", round(lr_acc*100,2), "%")
```

Logistic Regression Accuracy : 25.57 %

```
In [50]: #decision tree:
from sklearn.tree import DecisionTreeClassifier
dt = DecisionTreeClassifier(random_state=42)
dt.fit(X_train, y_train)
dt_pred = dt.predict(X_test)
```

```
In [51]: #accuracy:
dt_acc = accuracy_score(y_test, dt_pred)
print("Decision Tree Accuracy :", round(dt_acc*100,2), "%")
```

Decision Tree Accuracy : 15.47 %

```
In [52]: #Random forest:
from sklearn.ensemble import RandomForestClassifier
rf = RandomForestClassifier(n_estimators=200,random_state=42)
rf.fit(X_train, y_train)
rf_pred = rf.predict(X_test)
```

```
In [53]: #accuarcy:
```

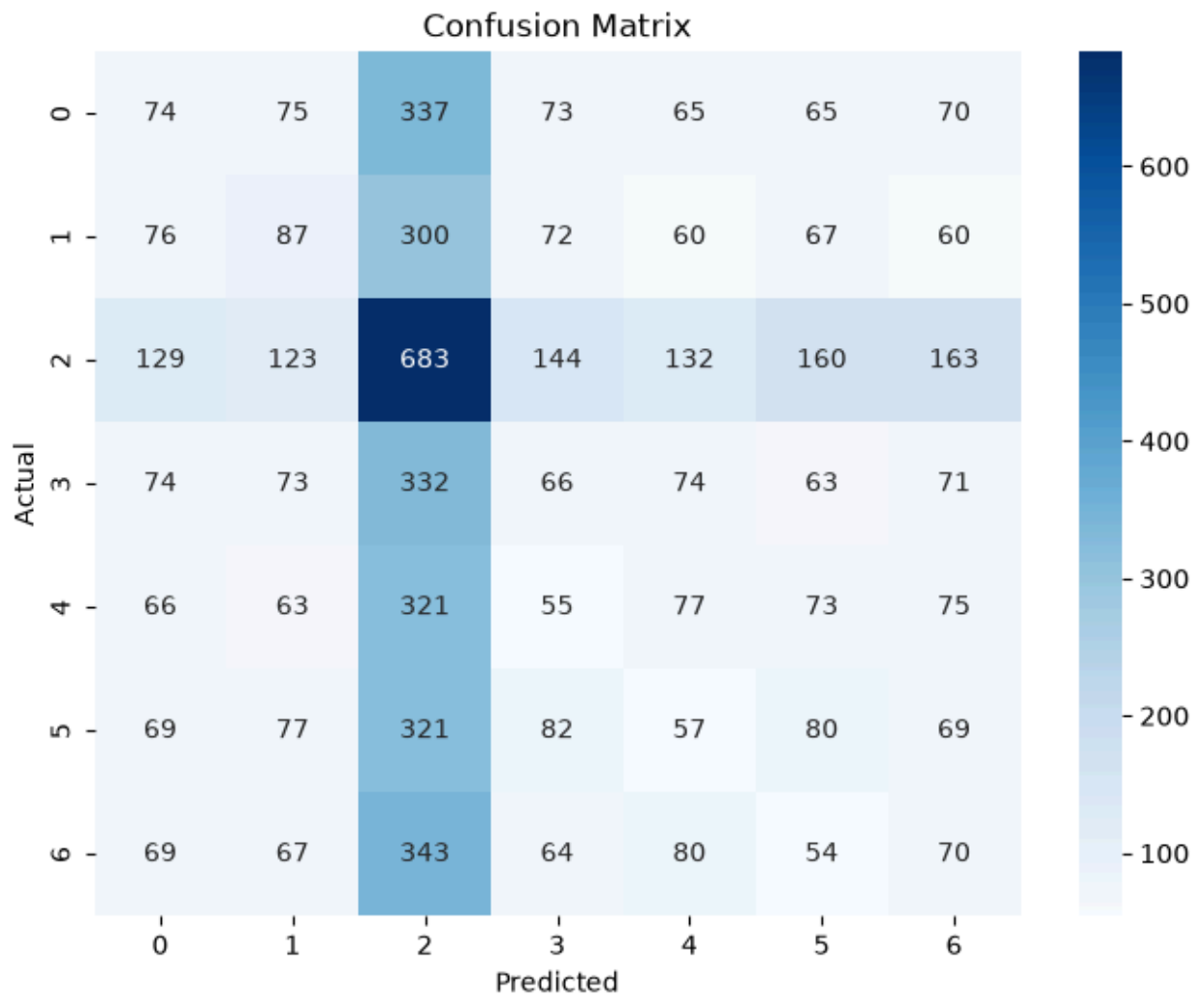
```
rf_acc = accuracy_score(y_test, rf_pred)
print("Random Forest Accuracy :", round(rf_acc*100,2), "%")
```

Random Forest Accuracy : 18.95 %

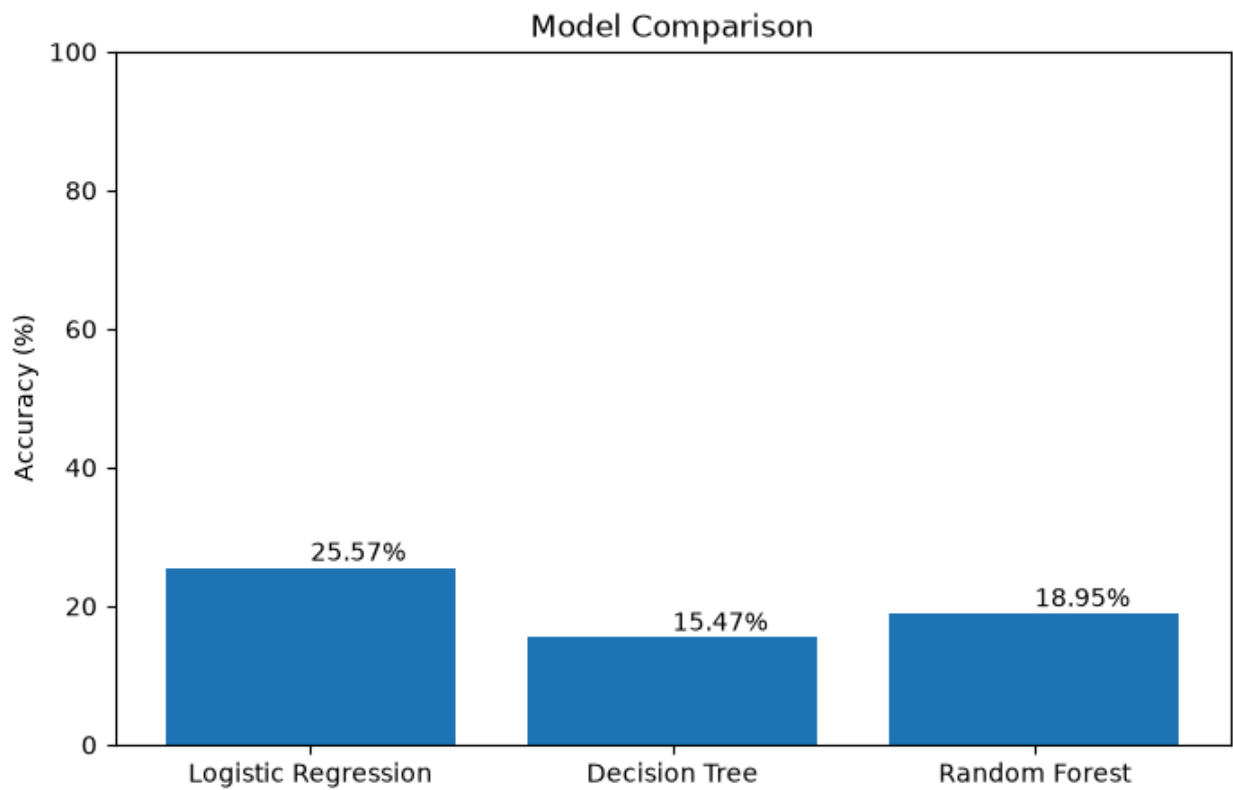
```
In [54]: #clasifying report:
from sklearn.metrics import classification_report
print(classification_report(y_test, rf_pred))
```

	precision	recall	f1-score	support
0	0.13	0.10	0.11	759
1	0.15	0.12	0.14	722
2	0.26	0.45	0.33	1534
3	0.12	0.09	0.10	753
4	0.14	0.11	0.12	730
5	0.14	0.11	0.12	755
6	0.12	0.09	0.11	747
accuracy			0.19	6000
macro avg	0.15	0.15	0.15	6000
weighted avg	0.17	0.19	0.17	6000

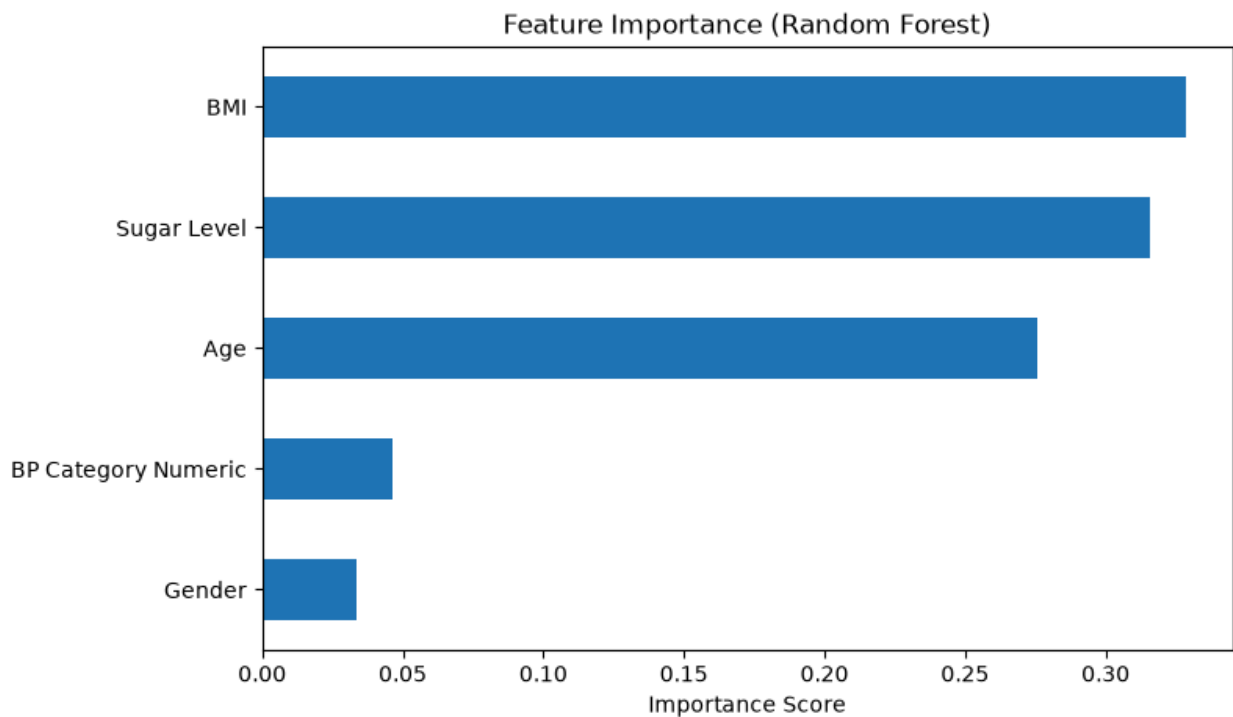
```
In [58]: #Confusion matrix:
from sklearn.metrics import confusion_matrix
cm = confusion_matrix(y_test, rf_pred)
plt.figure(figsize=(8,6))
sns.heatmap(cm,annot=True,fmt='d',cmap='Blues')
plt.title("Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```



```
In [59]: #compare model accuracy:
models = ['Logistic Regression','Decision Tree','Random Forest']
accuracy = [lr_acc*100,dt_acc*100,rf_acc*100]
plt.figure(figsize=(8,5))
plt.bar(models, accuracy)
plt.ylabel("Accuracy (%)")
plt.title("Model Comparison")
plt.ylim(0,100)
for i,v in enumerate(accuracy):
    plt.text(i,v+1,f"{v:.2f}%")
plt.show()
```



```
In [60]: #important features:
importance = pd.Series(rf.feature_importances_, index=X.columns)
importance.sort_values().plot(kind='barh',figsize=(8,5))
plt.title("Feature Importance (Random Forest)")
plt.xlabel("Importance Score")
plt.show()
```




```
In [61]: #prediction of new patients:
new_patient = pd.DataFrame({'Age': [45], 'Gender': [1], 'BP Category Numeric': [
new_patient = new_patient[rf.feature_names_in_]
prediction = rf.predict(new_patient)
predicted_disease = le_disease.inverse_transform(prediction)
print("Predicted Disease:", predicted_disease[0])
```

Predicted Disease: Hypertension

```
In [62]: import joblib
joblib.dump(rf, "Healthcare_RF_Model.pkl")
print("Random Forest model saved successfully!")
```

Random Forest model saved successfully!

```
In [2]: #conclusions:
#1. Diabetes dominates the disease distribution

#Out of all 30,000 patients, Diabetes turned out to be the single most common
#dataset. The other six disease categories (Flu, Healthy, Heart Disease, Hyper
#was more than double the size of any other single group. This class imbalance
#learning results.

#2. Gender and age look almost randomly spread out, not naturally distributed

#Gender was close to a 50-50 split, with Male patients making up about 50.5% (
#so there's no gender group that dominates the dataset. Age was even more inte
#histogram came out looking pretty flat across that whole range instead of the
#people clustered in adult/middle age). This basically means age in this datas
#randomly spread than what real-world health data usually looks like.

#3. Age, BMI, sugar level, and BP don't show any real relationship with each o

#I checked the correlation heatmap between Age, Sugar Level, BMI, and Blood Pr
#relationship – all the correlation values came out very close to 0. So based
#to go with higher BMI or higher sugar level in this data – the numbers just c
#a connection that isn't really there.

#4. No disease group stands out in terms of BMI, sugar level, or blood pressur

#Looking at the boxplots comparing BMI, Sugar Level, and Blood Pressure Catego
#lot – the medians and spreads look pretty similar from one disease to the nex
#higher BMI" or "that disease is linked to higher sugar levels" – visually, no
#parameters. I think it's more honest to report this as a lack of a clear risk

#5. The ML models struggled – Logistic Regression had the highest raw accuracy

#Three models were trained to predict Disease from Age, Gender, BP Category, S
#Random Forest scored 18.95%, and Decision Tree scored 15.47%. On the surface
#as just always guessing Diabetes (which makes up 25.6% of the data) – so it c
```

*#on the class imbalance. The Random Forest's classification report backs this
#and f1-score (0.33), while every other disease had precision and recall sitti
#problem. In terms of feature importance, Age, Sugar Level, and BMI came out a
#and BP Category contributed the least. Overall, though, none of the three mod
#relationship between these features and the disease outcome – the honest take
#to work with.*

In []: